

Manual do Utilizador (MU) — Analisador Semântico

Índice

- 1. [Objetivo](#)
- 2. [O que é análise semântica](#)
- 3. [Fluxo geral do compilador](#)
- 4. [Como usar](#)
- 5. [Tipos de erros semânticos detectados](#)
- 6. [Relatório de saída](#)
- 7. [Exemplos de código](#)
- 8. [Estrutura de ficheiros](#)
- 9. [Limitações atuais](#)
- 10. [Glossário](#)

1. Objetivo

Este documento descreve como usar o **analisador semântico** do compilador C++. O analisador semântico recebe a Árvore de Sintaxe Abstrata (AST) produzida pelo parser e **valida a correcta utilização de tipos, escopos e declarações** do programa.

O analisador semântico verifica se o código-fonte está semanticamente correcto de acordo com as regras da linguagem, detectando erros como variáveis não declaradas, declarações duplicadas, incompatibilidade de tipos e outros.

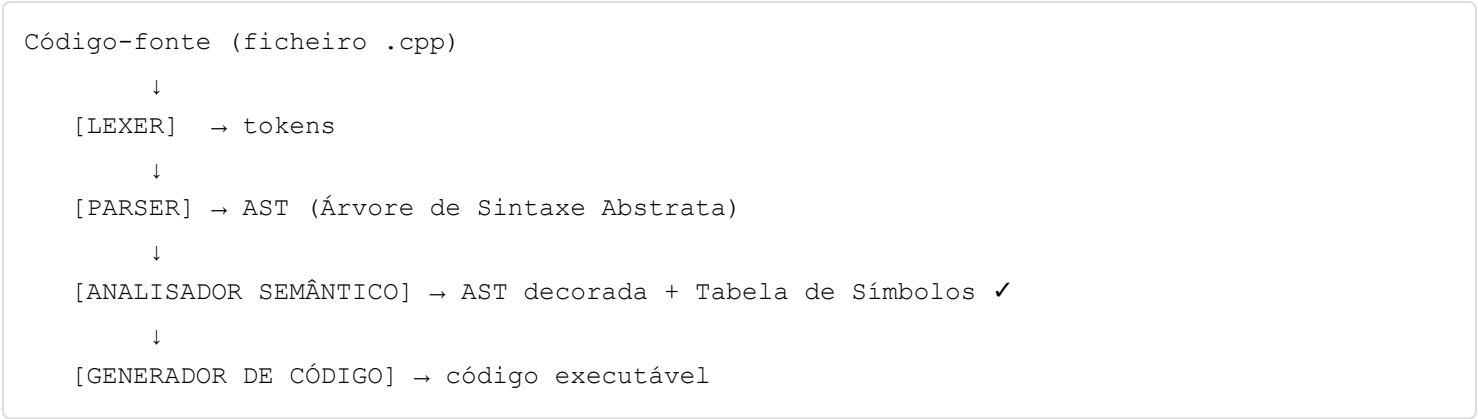
2. O que é análise semântica

A análise semântica é a **terceira fase** do compilador. Enquanto o parser valida a *estrutura* do código (se a sintaxe está correcta), o analisador semântico valida o *significado*:

Fase	O que valida	Exemplo de erro detectado
Lexer	Caracteres válidos	@x (caractere inválido)
Parser	Estrutura gramatical	Falta de ;
Semântico	Significado do código	Variável não declarada, tipos incompatíveis

Exemplo: `int x = "texto";` é sintacticamente válido (parser aceita), mas semanticamente errado (não se pode atribuir uma string a um int).

3. Fluxo geral do compilador



Estado actual: Fase 1 (Lexer), Fase 2 (Parser com AST) e Fase 3 (Analisador Semântico) implementadas.

4. Como usar

4.1 Pré-requisitos

- Rust e Cargo instalados
- Ficheiros `.cpp` na pasta `src/files/`

4.2 Execução

No directório raiz do projecto:

```
cargo run
```

O compilador processa **todos os ficheiros** `.cpp` na pasta `src/files/` e gera um ficheiro `relatorio.txt` com os resultados.

4.3 Formato do relatório

O relatório contém, para cada ficheiro:

1. **Nome do ficheiro**
2. **Resultado:** `SUCESSO` ou `ERRO SEMANTICO`
3. **Se SUCESSO:** AST decorada (com tipos inferidos) + Tabela de Símbolos
4. **Se ERRO:** Mensagem descritiva do erro com número da linha

5. Tipos de erros semânticos detectados

5.1 Variável não declarada

Quando se usa uma variável que não foi declarada no escopo actual ou em qualquer escopo ancestral.

```
Variavel 'y' nao declarada na linha 6
```

5.2 Variável declarada duas vezes no mesmo escopo

Quando se declara uma variável com o mesmo nome duas vezes no mesmo escopo.

```
Variavel 'x' ja declarada na linha 5, redeclarada na linha 6
```

5.3 Inicialização incompatível

Quando o tipo do valor inicializado é incompatível com o tipo declarado (sem conversão implícita possível).

```
Inicializacao incompativel na linha 9: 'int' = 'string'
```

5.4 Atribuição incompatível

Quando se atribui um tipo incompatível a uma variável já declarada.

```
Atribuicao incompativel na linha 9: 'int' = 'string'
```

5.5 Tipos incompatíveis em operações binárias

Quando se usam operandos de tipos incompatíveis num operador.

```
Operador '+' incompativel com 'string' e 'int' na linha 5
```

5.6 Função não declarada

Quando se chama uma função que não existe na tabela de símbolos.

```
Funcao 'foo' nao declarada na linha 10
```

5.7 Argumentos incompatíveis em chamada de função

Quando o número ou tipo dos argumentos não corresponde à assinatura da função.

```
Funcao 'soma' na linha 12 espera 2 argumentos, mas 1 foram passados
```

```
Argumento 1 da funcao 'soma' na linha 10: esperado 'int', obtido 'string'
```

5.8 Condição inválida em estruturas de controlo

Quando a condição de um `if`, `while`, `do-while` ou `for` não é `bool` ou `int`.

```
Condicao do if deve ser 'bool' ou 'int', encontrou 'string' na linha 14
```

5.9 Break/Continue fora de contexto

Quando `break` aparece fora de um loop ou `switch`, ou `continue` fora de um loop.

```
Break na linha 11 fora de loop ou switch
```

```
Continue na linha 8 fora de loop
```

5.10 Tipo de retorno incompatível

Quando o tipo da expressão de `return` não é compatível com o tipo de retorno da função.

```
Tipo de retorno incompativel na linha 19: funcao retorna 'int', mas expressao e 'string'
```

5.11 Declaração duplicada de construtor

Quando uma classe tem dois construtores com o mesmo nome.

```
Construtor 'Pessoa' ja declarado na linha 17, redeclarado na linha 23
```

5.12 Índice de array não inteiro

Quando o índice ou dimensão de um array não é do tipo `int`.

```
Indice do array deve ser 'int', encontrou 'double' na linha 7
```

6. Relatório de saída

6.1 Ficheiro de saída

O relatório é escrito em `relatorio.txt` na raiz do projecto.

6.2 Exemplo de relatório (sucesso)

```
--- test_ok.cpp ---
```

Resultado: SUCESSO

AST Decorada (com `inferred_type`):

Program

Include

Literal (`stdio.h`)

FunctionDecl

Type (`int`)

Identifier (`main`)

ParamList

Block

VarDecl

Type (`int`)

Identifier (`x`)

Literal (`10`)

Return

Literal (`0`)

Tabela de Simbolos:

Escopo 0 (nivel 0):

main | tipo: `Int("int")` | cat: `Function` | linha: 3 | addr: 0 | bytes: 0

Escopo 1 (nivel 1):

x | tipo: `Int("int")` | cat: `Variable` | linha: 5 | addr: 0 | bytes: 4

6.3 Exemplo de relatório (erro)

```
--- test_undeclared.cpp ---
```

Resultado: ERRO SEMANTICO

Variavel 'y' nao declarada na linha 6

6.4 Interpretação da Tabela de Símbolos

Campo	Significado
tipo	Tipo inferido (<code>Int("int")</code> , <code>Float("float")</code> , etc.)
cat	Categoria: <code>Variable</code> , <code>Function</code> , <code>Array</code> , <code>Parameter</code> , <code>Class</code>
linha	Linha onde foi declarada
addr	Endereço de memória relativo ao escopo
bytes	Tamanho em bytes
params	Lista de tipos dos parâmetros (apenas para funções)

6.5 Escopos

Os escopos são numerados e organizados em hierarquia:

Escopo 0 (nivel 0): ← Global (funções, classes)

Escopo 1 (nivel 1): ← Corpo da função

Escopo 2 (nível 2): ← Bloco dentro da função (if, while, for)
Escopo 3 (nível 3): ← Bloco aninhado dentro de outro bloco

7. Exemplos de código

7.1 Código sem erros

```
#include <stdio.h>

int soma(int a, int b) {
    return a + b;
}

int main() {
    int x = 10;
    float y = 3.14;
    int resultado = soma(x, 20);
    if (x > 5) {
        int temp = x;
    }
    return 0;
}
```

Resultado: SUCESSO — todos os tipos são compatíveis, todas as variáveis estão declaradas.

7.2 Código com erro: variável não declarada

```
int main() {
    int x = 10;
    y = 20; // y não foi declarada
    return 0;
}
```

Resultado: ERRO — "Variavel 'y' nao declarada na linha 6"

7.3 Código com erro: declaração duplicada

```
int main() {
    int x = 10;
    int x = 20; // x já foi declarada
    return 0;
}
```

Resultado: ERRO — "Variavel 'x' ja declarada na linha 5, redeclarada na linha 6"

7.4 Código com erro: tipo incompatível

```
int main() {  
    int x = 10;  
    x = "erro";    // não se pode atribuir string a int  
    return 0;  
}
```

Resultado: ERRO — "Atribuicao incompativel na linha 8: 'int' = 'string'"

7.5 Código com erro: argumentos de função

```
int soma(int a, int b) {  
    return a + b;  
}  
  
int main() {  
    int x = soma(10);    // espera 2 argumentos, recebe 1  
    return 0;  
}
```

Resultado: ERRO — "Funcao 'soma' na linha 10 espera 2 argumentos, mas 1 foram passados"

7.6 Código com erro: condição em estrutura de controlo

```
int main() {  
    if ("texto") {    // condição deve ser bool ou int  
        int x = 1;  
    }  
    return 0;  
}
```

Resultado: ERRO — "Condicao do if deve ser 'bool' ou 'int', encontrou 'string' na linha 4"

7.7 Código com sucesso: for com declaração

```
int main() {  
    for (int i = 0; i < 10; i = i + 1) {  
        int temp = i;  
    }  
    return 0;  
}
```

Resultado: SUCESSO — `i` é visível apenas dentro do body do for.

7.8 Código com sucesso: return em if aninhado

```
int soma(int a, int b) {  
    if (a > 0) {
```

```
        return a + b;
    }
    return a - b;
}
```

Resultado: SUCESSO — o return dentro do if está correctamente dentro da função.

8. Estrutura de ficheiros

```
compiler_cplusplus/
├─ src/
│   ├─ main.rs           ← ponto de entrada
│   ├─ lexer.rs          ← tokenização
│   ├─ token.rs          ← definição de tokens
│   ├─ ast.rs            ← tipos de nós da AST
│   ├─ analise_sintatica.rs ← parser LL(1) com construção de AST
│   ├─ analise_semantica.rs ← analisador semântico
│   ├─ scope.rs          ← tipos de escopo e tabela de símbolos
│   ├─ utils.rs          ← funções auxiliares
│   └─ files/            ← ficheiros de teste .cpp
├─ documentos/
│   └─ pp2-analise-sintatica/
│       └─ AST.md        ← especificação da AST
├─ mds/
│   ├─ MU.md             ← manual do utilizador (lexer)
│   ├─ MP.md             ← manual do programador (lexer)
│   ├─ MU_PARSER.md      ← manual do utilizador (parser)
│   ├─ MP_PARSER.md      ← manual do programador (parser)
│   ├─ MU_SEMANTICO.md   ← este ficheiro
│   └─ MP_SEMANTICO.md   ← manual do programador (semântico)
├─ relatorio.txt         ← relatório de saída gerado
└─ Cargo.toml
```

9. Limitações atuais

1. **Subconjunto de C++** — não suporta templates, namespaces, herança múltipla
 2. **Sem conversão implícita completa** — $\text{int} \leftrightarrow \text{float}$ é permitido, mas $\text{float} \rightarrow \text{int}$ não é convertido automaticamente
 3. **Escopo de classes simplificado** — membros de classe são tratados como declarações simples
 4. **Sem verificação de overload** — funções com o mesmo nome mas assinaturas diferentes não são suportadas
 5. **For-loop com initializer complexo** — `for (int i = 0, j = 1; ...)` não é suportado (apenas uma variável)
 6. **Sem verificação de ponteiros/referências** — operadores `*` e `&` são aceitos mas não validados semanticamente
-

10. Glossário

Termo	Significado
AST	Árvore de Sintaxe Abstrata — representação hierárquica do código
Escopo	Contexto onde um nome é válido (ex: corpo de função, bloco if)
Tabela de Símbolos	Estrutura que armazena informações sobre nomes declarados
Inferência de tipo	Processo de deduzir o tipo de uma expressão a partir dos seus operandos
Promoção de tipo	Conversão implícita entre tipos compatíveis (ex: int→float)
Lookup	Busca de um nome subindo a hierarquia de escopos
Variable Shadowing	Declaração de uma variável com o mesmo nome numa escopo interno
Pending Parameters	Mecanismo para adiar a inserção de parâmetros no escopo correcto
BlockType	Tipo de constructo actual (Function, For, While, etc.) usado para validação
